```
"""Finds the index of the first element >= k using recursive binary search.
Returns -1 if no such element is found.
"""


def find_first_greater_or_equal_recursive(nums: list, k: int) -> int:
    """Return the first index where nums[i] >= k using recursive binary search."""

    # Base case: If the list is empty, then the value wasn't found
    if len(nums) == 0:
        return -1

    # Get mid index using floor division
    mid = len(nums) // 2

    # If the mid value is greater or equal to the target, look at the left side of the list
    # for a smaller value
    if nums[mid] >= k:
        result = find_first_greater_or_equal_recursive(nums[:mid], k)

        # If the left half doesn't have any greater or equal values, then the mid is the first value
        if result == -1:
            return mid
        # Otherwise, return the earlier index found in the left half
        return result

    # Otherwise search the right side of the list
    else:
        result = find_first_greater_or_equal_recursive(nums[mid + 1:], k)

        # If the right side has no greater or equal element, then no valid element exists
        if result == -1:
            return -1

        # result is the index within the sliced list
        # mid + 1 is where the slice starts in the original list
        return mid + 1 + result



# Test Cases
print("--- Question 1: Recursive Lower Bound ---")
nums1 = [2, 5, 8, 12, 16]
print(f"List: {nums1}")
print(f"Searching for k=1, result: {find_first_greater_or_equal_recursive(nums1, 1)}")  # Expected: 0
print(f"Searching for k=2, result: {find_first_greater_or_equal_recursive(nums1, 2)}")  # Expected: 0
print(f"Searching for k=9, result: {find_first_greater_or_equal_recursive(nums1, 9)}")  # Expected: 3
print(f"Searching for k=16, result: {find_first_greater_or_equal_recursive(nums1, 16)}")  # Expected: 4
print(f"Searching for k=18, result: {find_first_greater_or_equal_recursive(nums1, 18)}")  # Expected: -1 (Not found)
print("-" * 20)

nums2 = [10, 20, 30, 40, 40, 50]
print(f"List: {nums2}")
print(f"Searching for k=25, result: {find_first_greater_or_equal_recursive(nums2, 25)}")  # Expected: 2
print(f"Searching for k=40, result: {find_first_greater_or_equal_recursive(nums2, 40)}")  # Expected: 3
print(f"Searching for k=60, result: {find_first_greater_or_equal_recursive(nums2, 60)}")  # Expected: -1
print("-" * 20)

nums3 = [10, 20, 20, 20, 30, 50]
print(f"List: {nums3}")
print(f"Searching for k=20, result: {find_first_greater_or_equal_recursive(nums3, 20)}")  # Expected: 1
print(f"Searching for k=25, result: {find_first_greater_or_equal_recursive(nums3, 25)}")  # Expected: 4
print(f"Searching for k=30, result: {find_first_greater_or_equal_recursive(nums3, 30)}")  # Expected: 4
print("-" * 20)


class Node:
    """A single node in a BST."""

    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

```
class BinarySearchTree:
    """A Binary Search Tree data structure."""

    def __init__(self):
        self.root = None

    # --- Question 2: Insertion and Search ---
    def insert(self, value):
        """Insert a value into the BST."""
        new_node = Node(value)
        if self.root is None:
            self.root = new_node
            return

        current = self.root
        while True:
            # Smaller values belong on the left; larger values belong on the right.
            if value < current.value:
                if current.left is None:
                    current.left = new_node
                    return
                current = current.left
            elif value > current.value:
                if current.right is None:
                    current.right = new_node
                    return
                current = current.right
            else:
                # Ignore duplicates so values remain strictly ordered.
                return

    def search(self, value):
        """Search for a value in the BST."""
        current = self.root
        while current is not None:
            if value == current.value:
                return True
            # BST ordering lets the search skip the subtree that cannot contain the value.
            current = current.left if value < current.value else current.right
        return False

    # --- Question 3: Deletion and Search ---
    def delete(self, value):
        """Delete a value from the BST while preserving the BST property."""
        # Save the returned root because deleting may replace the root node.
        self.root = self._delete_recursive(self.root, value)

    def _delete_recursive(self, node, value):
        if node is None:
            return None

        if value < node.value:
            node.left = self._delete_recursive(node.left, value)
        elif value > node.value:
            node.right = self._delete_recursive(node.right, value)
        else:
            # For zero or one child, the child can take this node's place directly.
            if node.left is None:
                return node.right
            if node.right is None:
                return node.left

            # Replace a two-child node with the smallest value in its right subtree.
            successor = node.right
            while successor.left is not None:
                successor = successor.left
            node.value = successor.value
            node.right = self._delete_recursive(node.right, successor.value)

        return node

    # --- Validation Method ---
    def is_valid_bst(self):
        """
        Check if the tree is a valid BST.
        Initializes the recursive check with no bounds.
        """
        return self._is_valid_bst_recursive(self.root, float("-inf"), float("inf"))
```

```
    def _is_valid_bst_recursive(self, node, min_val, max_val):
        """
        Recursive helper to validate the BST property.
        Checks if a node's value is within the specified bounds.
        """
        if node is None:
            return True

        # Check if the current node's value violates the BST property
        if not (min_val < node.value < max_val):
            return False

        # Recursively check the left subtree with updated max_val
        left_valid = self._is_valid_bst_recursive(node.left, min_val, node.value)

        # Recursively check the right subtree with updated min_val
        right_valid = self._is_valid_bst_recursive(node.right, node.value, max_val)
        return left_valid and right_valid


# Initialize an empty BST
bst = BinarySearchTree()
print("=== Question 2: Insertion and Search ===")

# Part 1: Insertion Test Cases
print("\n--- Part 1: Insertion Test Cases ---")
values_to_insert = [15, 8, 20, 5, 12, 18, 25, 7, 10, 16, 22, 28, 5]
for value in values_to_insert:
    bst.insert(value)

# Test a valid BST
print("Testing a valid BST:")
print(f"Is the current tree a valid BST? {bst.is_valid_bst()}")  # Expected: True

# Part 2: Search Test Cases
print("\n--- Part 2: Search Test Cases ---")
print(f"Search for 5: {bst.search(5)}")  # Expected: True (duplicate)
print(f"Search for 7: {bst.search(7)}")  # Expected: True (leaf)
print(f"Search for 12: {bst.search(12)}")  # Expected: True
print(f"Search for 13: {bst.search(13)}")  # Expected: False
print(f"Search for 22: {bst.search(22)}")  # Expected: True (right subtree)
print(f"Search for 28: {bst.search(28)}")  # Expected: True (leaf)
print("-" * 20)
print("\n=== Question 3: Deletion and Search ===")

# Case A: Delete a leaf node
print("\n--- Case A: Delete Leaf Node (7) ---")
bst.delete(7)
print(f"Search for 7 after deletion: {bst.search(7)}")  # Expected: False
print(f"Is the current tree a valid BST? {bst.is_valid_bst()}")  # Expected: True

# Case B: Delete a node with one child
print("\n--- Case B: Delete Node with One Child (18) ---")
bst.delete(18)
print(f"Search for 18 after deletion: {bst.search(18)}")  # Expected: False
print(f"Is the current tree a valid BST? {bst.is_valid_bst()}")  # Expected: True

# Case C: Delete a node with two children
print("\n--- Case C: Delete Node with Two Children (20) ---")
bst.delete(20)
print(f"Search for 20 after deletion: {bst.search(20)}")  # Expected: False
print(f"Is the current tree a valid BST? {bst.is_valid_bst()}")  # Expected: True

# Value not found: tree should remain valid
print("\n--- Delete Value Not Present (100) ---")
bst.delete(100)
print(f"Is the current tree a valid BST? {bst.is_valid_bst()}")  # Expected: True
print("-" * 20)
```

```
--- Question 1: Recursive Lower Bound ---
List: [2, 5, 8, 12, 16]
Searching for k=1, result: 0
Searching for k=2, result: 0
Searching for k=9, result: 3
Searching for k=16, result: 4
```

```
Searching for k=18, result: -1
--------------------
List: [10, 20, 30, 40, 40, 50]
Searching for k=25, result: 2
Searching for k=40, result: 3
Searching for k=60, result: -1
--------------------
List: [10, 20, 20, 20, 30, 50]
Searching for k=20, result: 1
Searching for k=25, result: 4
Searching for k=30, result: 4
--------------------
=== Question 2: Insertion and Search ===

--- Part 1: Insertion Test Cases ---
Testing a valid BST:
Is the current tree a valid BST? True

--- Part 2: Search Test Cases ---
Search for 5: True
Search for 7: True
Search for 12: True
Search for 13: False
Search for 22: True
Search for 28: True
--------------------

=== Question 3: Deletion and Search ===

--- Case A: Delete Leaf Node (7) ---
Search for 7 after deletion: False
Is the current tree a valid BST? True

--- Case B: Delete Node with One Child (18) ---
Search for 18 after deletion: False
Is the current tree a valid BST? True

--- Case C: Delete Node with Two Children (20) ---
Search for 20 after deletion: False
Is the current tree a valid BST? True

--- Delete Value Not Present (100) ---
Is the current tree a valid BST? True
--------------------
```